TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 2 · BLUE TEAM & DEFENCE

WEEK 07

IDS/IPS and Network Security Monitoring

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**7 of 12**	**2 — Blue Team & Defence**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 7: IDS/IPS and Network Security Monitoring**
  - Phase: Blue Team and Defence (Weeks 5–8)
  - Day 1 — IDS/IPS Fundamentals and Snort Architecture
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Writing Custom Snort Rules
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Suricata and Advanced Detection
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Network Traffic Analysis with tcpdump and AI
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 8**

Phase: Blue Team and Defence (Weeks 5–8) **Week 7 of 12**

WEEK 7: IDS/IPS AND NETWORK SECURITY MONITORING

Weekly Overview

This week you add network-based detection to your defensive toolkit. You will learn the difference between intrusion detection (IDS) and prevention (IPS), write custom Snort rules to detect web attacks, analyse live traffic with tcpdump, and enhance detection with AI-powered signature generation. By the end of this week, you will have a complete detection stack: SIEM (logs from Week 6) plus IDS (network traffic from Week 7).

How this week builds on Weeks 5–6

- Week 5 taught you web attack patterns (SQLi, XSS). Week 6 taught you to detect them in logs with Splunk. Now you will detect them on the wire with Snort — catching attacks even if logging is disabled.
- Your Week 3 tcpdump skills return for traffic analysis.

How this week sets up Week 8

- Week 8 is Digital Forensics and Incident Response. The network traffic captures you learn this week become forensic evidence. IDS alerts trigger incident response investigations.

Why this matters in a real security job

- Attackers delete logs. They disable agents. But network traffic cannot be deleted (if captured at the switch). IDS/IPS provides evidence that survives host compromise. Every mature SOC has network detection.

■ Authorisation Reminder

All packet captures and IDS rules this week target your own lab network traffic only. Sniffing networks without permission is illegal under Nigeria's Cybercrimes Act 2015.

DAY 1

IDS/IPS Fundamentals and Snort Architecture

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain the difference between IDS and IPS

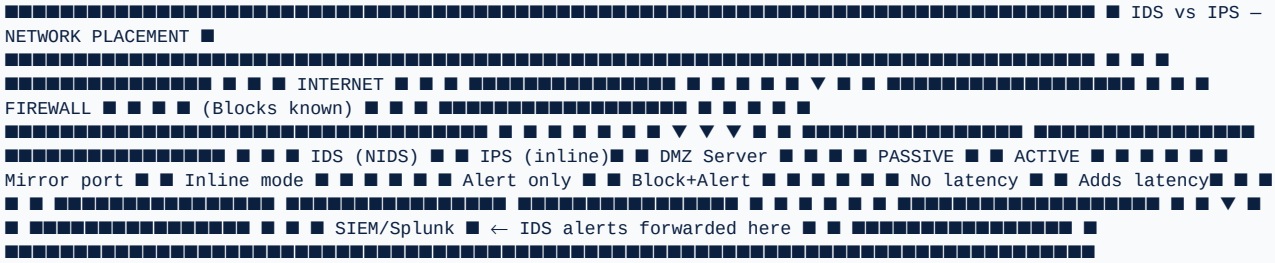
2. Distinguish between NIDS and HIDS

3. Describe Snort architecture and rule structure

IDS (INTRUSION DETECTION SYSTEM) / IPS (INTRUSION PREVENTION SYSTEM)

IDS monitors network traffic for suspicious activity and generates alerts. IPS adds the ability to automatically block malicious traffic in real-time — turning passive detection into active prevention.

IDS vs IPS — Network Placement



IDS vs IPS — Key Differences

Feature	IDS (Detection)	IPS (Prevention)
Position	Passive — mirror/span port	Inline — traffic passes through it
Action on threat	Alert only — no blocking	Alert AND block malicious traffic
Network impact	Zero — traffic not affected	Adds latency — needs tuning
False positive risk	Low risk — only alerts	High risk — false positives block legit traffic
Best use	Monitoring and visibility	Enforcement and prevention
Example tools	Snort (IDS mode), Zeek	Snort (IPS mode), Suricata, Palo Alto NGFW

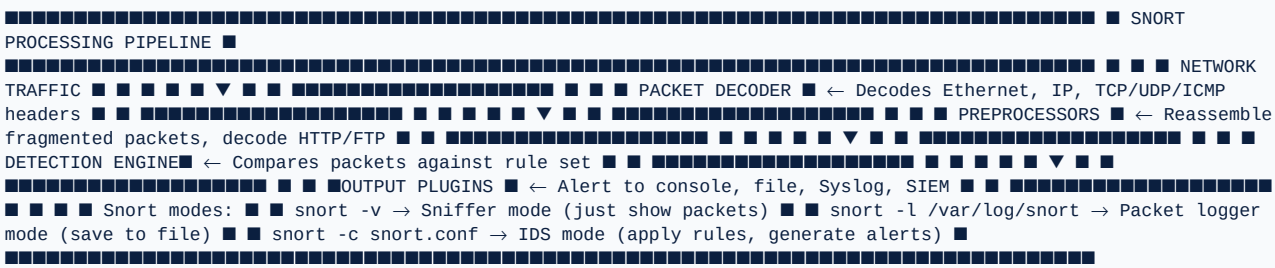
NIDS vs HIDS

Type	Where It Runs	What It Monitors	Best For

NIDS (Network-based)	Dedicated sensor/appliance on the network	All network traffic passing through a segment	Detecting network attacks (port scans, exploits, C2)
HIDS (Host-based)	Software agent on each endpoint	System calls, file changes, log entries on that host	Detecting insider threats, file tampering, malware
Hybrid	Both NIDS and HIDS deployed	Network traffic + host-level events	Complete coverage — what most mature SOCs use

Snort Architecture

Snort Processing Pipeline



Detection Methods — Signatures vs Anomaly vs Behaviour

Method	How It Works	Pros	Cons
Signature-based	Matches traffic against known attack patterns	Low false positives; fast; well understood	Cannot detect new/unknown attacks (zero-days)
Anomaly-based	Builds baseline; flags deviations	Detects new attacks; no signature needed	High false positives; needs tuning period
Behaviour-based	Models expected behaviour; flags unusual activity	Detects insider threats; context-aware	Complex to implement; resource intensive
Reputation-based	Checks IPs/domains against threat intel feeds	Quick wins on known bad actors	Only covers known threats

WHY THIS MATTERS

In 2021, the Oldsmar water treatment facility hack (Florida, USA) was detected because an employee noticed unusual mouse movements — the attacker was controlling the plant's HMI system remotely. No IDS was in place. A basic NIDS rule detecting unusual remote access protocols on industrial control networks would have alerted before the attacker increased sodium hydroxide levels to 111 times the safe limit. Network detection saves lives in critical infrastructure.

PRACTICE *(2 Hours)*

LAB 7.1.1 — Install and Configure Snort on Kali

Objective: Install Snort and verify it can capture network traffic in your lab.

Prerequisites: Kali Linux running; Metasploitable2 running on same network

Step 1 — Install Snort

BASH

```
sudo apt update && sudo apt install snort -y # Verify installation snort --version # During install you may be prompted for your network range # Enter your lab subnet: e.g. 192.168.1.0/24
```

Step 2 — Run Snort in Sniffer Mode

BASH

```
# Show all packets on the network interface sudo snort -v -i eth0 # Show packets with more detail (headers) sudo snort -vd -i eth0 # Show packet payload data sudo snort -vde -i eth0
```

What to observe: source/destination IPs, ports, protocol, TCP flags. Generate some traffic (ping Metasploitable, browse to DVWA) and watch Snort capture it.

Step 3 — Run Snort in IDS Mode with Community Rules

BASH

```
# Run Snort with default config and rules sudo snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort -A console # -c = config file # -i = network interface # -l = log directory # -A console = print alerts to screen
```

Step 4 — Generate Traffic and Watch Alerts

From another terminal, run an Nmap scan against Metasploitable:

BASH

```
sudo nmap -sS TARGET_IP
```

Watch the Snort console — you should see alerts for the port scan. This demonstrates Snort detecting Week 4 attack techniques.

■ ANALYSE THIS

Snort fired an alert for your Nmap scan. What rule triggered it? Look in /etc/snort/rules/ for the rule file. What does the rule syntax mean? Could an attacker evade this rule by using -T1 timing? How would you modify the rule to also catch slow scans?

BUILD *(2 Hours)*

BUILD BRIEF

Create an IDS/IPS Selection and Deployment Guide for a fictional client.

Scenario:

- A Nigerian fintech startup (50 employees, cloud-hosted web app, on-premises office network) has no IDS/IPS and has just had a security incident

Required sections:

- IDS vs IPS Decision: for their specific environment, recommend IDS or IPS and explain why

- NIDS vs HIDS: recommend deployment of one or both and justify
- Tool Selection: compare Snort vs Suricata vs commercial options (Palo Alto, Cisco) for their budget
- Network Placement Diagram: ASCII diagram showing where sensors go in their network
- Deployment Timeline: 30/60/90-day implementation plan
- Rule Management: how to keep signatures up-to-date; how to handle false positives

Format:

- Professional consulting recommendation report

DOCUMENT

Writing Prompt:

Write a Network Security Monitoring Policy for your organisation.

- Required: Purpose and scope, which network segments will be monitored, data retention requirements for packet captures, authorisation required to review captures (privacy implications), incident escalation when IDS fires, compliance alignment (NDPA — monitoring employee traffic has privacy implications)
- This is a governance document — it defines the rules for how IDS/IPS will be operated

DAY 2

Writing Custom Snort Rules

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Parse and understand the Snort rule syntax in full

2. Write custom Snort rules to detect SQLi, XSS, and port scans

3. Test rules against live traffic and tune to reduce false positives

SNORT RULE

is a pattern-matching instruction that tells Snort what network traffic to alert on. Each rule defines: the action to take, the traffic to inspect, and the pattern to match.

Snort Rule Anatomy

```
■■■
```

threshold	threshold: type limit, track by_src, count 5, seconds 60;	Rate limiting	Detect scans/brute force
detection_filter	detection_filter: track by_src, count 10, seconds 5;	Minimum events before alert	Reduce single-packet FPs
sid	sid:1000001;	Rule ID	Must be unique per rule
rev	rev:1;	Revision number	Track rule changes

Building a SQLi Detection Rule Step by Step

SNORT RULES — SQL INJECTION DETECTION

```
# Basic SQLi detection — catches 'union select' in HTTP traffic alert tcp any any -> $HTTP_SERVERS
$HTTP_PORTS ( msg:"TECHRISE SQLi Detected - UNION SELECT"; flow:to_server,established;
http_method:GET; http_uri; content:"union"; nocase; content:"select"; nocase; distance:0;
classtype:web-application-attack; sid:1000001; rev:1; ) # Authentication bypass SQLi alert tcp any any
-> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE SQLi Auth Bypass - OR 1=1"; flow:to_server,established;
http_uri; pcre:"/(%27|')\s*(or|and)\s*(1=1|'1'='1')/i"; classtype:web-application-attack;
sid:1000002; rev:1; ) # SQL comment injection alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (
msg:"TECHRISE SQLi Comment - Double Dash"; flow:to_server,established; http_uri; content:"--";
http_uri; content:"'"; http_uri; classtype:web-application-attack; sid:1000003; rev:1; )
```

XSS and Scan Detection Rules

SNORT RULES — XSS AND SCAN DETECTION

```
# XSS detection — script tag in URL alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE XSS
- Script Tag in URI"; flow:to_server,established; http_uri; content:"script"; nocase;
classtype:web-application-attack; sid:1000004; rev:1; ) # Port scan detection alert tcp any any ->
$HOME_NET any ( msg:"TECHRISE Port Scan Detected"; flags:S; threshold: type both, track by_src, count
20, seconds 5; classtype:attempted-recon; sid:1000005; rev:1; ) # Nikto web scanner signature alert
tcp any any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"TECHRISE Web Scanner - Nikto Detected";
flow:to_server,established; http_header; content:"Nikto"; nocase; classtype:attempted-recon;
sid:1000006; rev:1; )
```

WHY THIS MATTERS

During the 2014 Target breach, attackers used a remote access tool that communicated over standard HTTP to blend in with normal traffic. The traffic contained distinctive patterns that a well-tuned Snort rule would have caught — but Target's security team had disabled the automated blocking feature of their IPS due to too many false positives. Tuning detection rules is just as important as writing them.

PRACTICE *(2 Hours)*

LAB 7.2.1 — Deploy and Test Custom Snort Rules

Objective: Write, deploy, and validate your custom detection rules.

Prerequisites: Snort installed; DVWA running; Week 5 attack tools available

Step 1 — Create Your Local Rules File

BASH

```
sudo nano /etc/snort/rules/local.rules # Add your SQLi detection rules # Save and exit
```

Step 2 — Verify Rule Syntax

BASH

```
# Test config without starting snort sudo snort -T -c /etc/snort/snort.conf # Expected: Snort
successfully validated the configuration!
```

Step 3 — Start Snort with Your Rules

BASH

```
sudo snort -c /etc/snort/snort.conf -i eth0 -l /var/log/snort -A console
```

Step 4 — Trigger Your Rules — SQLi

In a second terminal, run a SQLi attack against DVWA:

BASH

```
curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--+&Submi
t;=Submit' # URL encoded version: curl
'http://localhost/dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271
```

Watch the Snort console — you should see your SQLi alert fire. Record the exact alert output.

Step 5 — Trigger Port Scan Rule

BASH

```
sudo nmap -sS localhost
```

Watch for the port scan detection alert. Note the threshold behaviour — how many SYNs before it fires?

Step 6 — Test False Positive Rate

Navigate to DVWA normally (legitimate user browsing). Does your SQLi rule fire on normal traffic? If yes — tune the rule to reduce false positives by adding more specific content matches or using the http_uri modifier.

■ ANALYSE THIS

Your SQLi rule fires on legitimate traffic when a user submits a form containing the word 'union' (e.g., a trade union membership form). How would you modify the rule to only match SQL-specific patterns? What additional context (HTTP method, content type, request structure) would help distinguish a SQLi attempt from legitimate input?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Snort Rule Library for your SOC — a curated set of custom rules covering your threat landscape.

Required rules (minimum 10, all tested):

- SQL injection — UNION SELECT, auth bypass, time-based blind
- XSS — script tag, img onerror, SVG onload (use entity encoding in rule content)
- Directory traversal — ../../etc/passwd pattern
- Web scanner fingerprints — Nikto, Gobuster, sqlmap User-Agent signatures
- Port scan detection with threshold
- SSH brute force — SYN flood on port 22
- Metasploit Meterpreter communication pattern
- Your choice: one additional rule for a threat relevant to Nigerian fintech

For each rule include:

- Rule syntax, what it detects, false positive risk, tuning notes, MITRE ATT&CK; technique ID

Format:

- Operational rule library document — each rule on its own entry, professionally documented

DOCUMENT

Writing Prompt:

Write a False Positive Management Procedure for your IDS/IPS deployment.

- Required: Definition of false positive vs true positive vs false negative, process for reporting and documenting FPs, decision criteria (when to tune a rule vs when to accept the FP), escalation path when tuning causes a true positive to be missed, metrics to track FP rate over time
- Include a specific example: your SQLi rule fires on legitimate form submissions containing 'union' — walk through the full tuning process

DAY
3

Suricata and Advanced Detection

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Explain key differences between Snort and Suricata

2. Install Suricata and run it with the Emerging Threats ruleset

3. Understand multi-threading and performance advantages of Suricata

SURICATA

is a high-performance, open-source Network Security Monitoring (NSM) engine capable of real-time intrusion detection, inline intrusion prevention, network security monitoring, and offline pcap processing.

Snort vs Suricata — Detailed Comparison

Feature	Snort 3	Suricata
Performance	Single-threaded (Snort 2) / Multi-threaded (Snort 3)	Natively multi-threaded — uses all CPU cores
Protocol support	TCP/UDP/ICMP + HTTP, FTP, SMTP decoders	Deep protocol inspection: HTTP, TLS, DNS, SSH, SMB, SMTP, FTP
TLS inspection	Limited without pfSense	Built-in TLS/SSL inspection with JA3 fingerprinting
File extraction	Requires additional config	Built-in file extraction from HTTP, SMTP, FTP, SMB
Output formats	Unified2, alert files	JSON (EVE format) — native SIEM integration
Rule compatibility	Snort rules	Compatible with Snort rules + Suricata-specific rules
Emerging Threats rules	Supported	Primary target — Emerging Threats Pro/Open
PCAP processing	Offline processing	Offline + stream processing
Best for	Learning IDS fundamentals	Production IDS/IPS deployments

Suricata's Unique Capabilities

- JA3/JA3S TLS fingerprinting: identifies malware by its TLS handshake pattern — even when traffic is encrypted
- File extraction: automatically extracts files transferred over HTTP, FTP, SMTP, SMB — enables malware analysis
- Protocol anomaly detection: detects malformed protocol headers that indicate evasion attempts

- Lua scripting: complex detection logic written in Lua for advanced threat detection
- EVE JSON output: every alert and network event logged in JSON — perfect for Splunk ingestion
- Netflow/IPFIX: generates flow records from captured traffic — enables long-term traffic analysis

Emerging Threats Ruleset

- Emerging Threats Open (ET Open): free, community-maintained ruleset — 40,000+ rules updated daily
- Covers: malware C2 traffic, exploit attempts, scanning, coin mining, botnet communication
- ET Pro: commercial version with faster update cycle and more rules
- Download and update: suricata-update (built-in tool for managing rulesets)
- Rule categories: emerging-exploit.rules, emerging-malware.rules, emerging-scan.rules, emerging-web.rules

Suricata EVE JSON Output

```
■■■
```

Step 3 — Run Suricata in IDS Mode

BASH

```
# Start Suricata with ET rules sudo suricata -c /etc/suricata/suricata.yaml -i eth0 # Watch the EVE JSON log in real-time sudo tail -f /var/log/suricata/eve.json | python3 -m json.tool | head -100
```

Step 4 — Generate Attack Traffic

Run your Week 5 SQLi attacks against DVWA, then run an Nmap scan. Watch the EVE JSON log for alerts.

Step 5 — Forward Suricata Alerts to Splunk

BASH

```
# In Splunk: Settings → Data Inputs → Files & Directories # File: /var/log/suricata/eve.json # Sourcetype: suricata (or _json) # SPL to view IDS alerts index=dvwa sourcetype=suricata event_type=alert | table _time, src_ip, dest_ip, alert.signature, alert.severity | sort -_time
```

This is the SIEM + IDS integration: Splunk now shows both log-based and network-based detections on one dashboard.

■ ANALYSE THIS

Suricata detected your SQLi attack 40ms after the packet hit the network. Your Splunk alert (based on Apache logs) fired 60 seconds later (log flush interval). In a real attack, what can an attacker accomplish in 60 seconds? What is the operational benefit of combining network-based and log-based detection?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Suricata Integration Guide — how to connect Suricata to your Splunk SIEM.

Required sections:

- Architecture diagram: shows Suricata → EVE JSON → Splunk Universal Forwarder → Splunk Indexer → Dashboard
- Installation steps (tested, exact commands)
- EVE JSON field reference: table of key fields and their security meaning
- Splunk sourcetype configuration for suricata
- 5 SPL queries that use Suricata + Splunk data together
- Dashboard panel designs for IDS alerts: top signatures, top src IPs, alert severity distribution, timeline

Bonus:

- Add a correlation search that fires when the same IP appears in both Suricata alerts AND auth.log failures

DOCUMENT

Writing Prompt:

Write a Comparative Analysis: Signature-Based vs AI-Powered Detection.

- Required: How signature-based detection works, how AI/ML anomaly detection works, 3 scenarios where signatures win (speed, precision, known threats), 3 scenarios where AI wins (zero-days, encrypted traffic, insider threats), your recommendation for a Nigerian fintech with a small security budget
- Use real examples: Snort/Suricata for signatures; Darktrace/Microsoft Sentinel for AI

DAY
4

Network Traffic Analysis with tcpdump and AI

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use tcpdump to capture and filter network traffic from the command line

2. Analyse PCAP files using Wireshark and tcpdump filters

3. Use AI to assist in identifying attack patterns in packet captures

TCPDUMP

is a command-line packet analyser that captures and displays network packets in real time. It is available on virtually every Unix/Linux system and is used when a GUI (like Wireshark) is not available — including on servers and network appliances.

tcpdump vs Wireshark

```
■■■
```

tcpdump -i eth0 'src net 192.168.1.0/24 and port 80'	Complex filter	Segment-specific web traffic

BPF (Berkeley Packet Filter) Syntax

TCPDUMP BPF FILTERS

```
# Capture traffic between two hosts tcpdump host 192.168.1.100 and 192.168.1.200 # Capture traffic to/from a network tcpdump net 192.168.1.0/24 # Capture specific port excluding noise tcpdump port 80 and not port 443 # Capture SYN packets only (start of connections — port scan indicator) tcpdump 'tcp[tcpflags] & (tcp-syn) != 0 and tcp[tcpflags] & (tcp-ack) = 0' # Capture large packets (potential data exfiltration) tcpdump greater 1400 # Capture HTTP POST requests (credential submission) tcpdump -i eth0 -A -s 0 port 80 | grep -i 'POST\|username\|password'
```

Wireshark Display Filters for Security Analysis

Filter	What It Shows	Attack Identified
http.request.method=="GET" && http.request.uri contains "union"	HTTP GET with SQLi	SQL injection attempts
http.response.code==500	HTTP 500 errors	Application errors (attack probing)
tcp.flags.syn==1 && tcp.flags.ack==0	SYN packets only	Port scanning activity
ftp.request.command=="PASS"	FTP password commands	FTP credential transmission
dns.qry.name contains ".onion"	Tor DNS queries	Darknet communication
icmp.type==8	ICMP echo requests	Ping sweep / network discovery
ssl.handshake.extensions_server_name	TLS SNI field	Which domains encrypted connections go to

AI-Assisted PCAP Analysis

- Export suspicious packets to text: tcpdump -r capture.pcap -A > capture.txt
- Feed excerpts to Claude: 'I captured this network traffic during a suspected intrusion. Analyse it and identify any attack patterns'
- Ask Claude to generate tcpdump filters: 'Write a tcpdump filter to capture only HTTP traffic containing SQL keywords'
- Ask Claude to explain Wireshark output: 'Explain what this TCP stream shows and whether it indicates a SQL injection attack'
- Never upload real customer data or PII to public AI tools — use sanitised/anonymised captures

WHY THIS MATTERS

In the aftermath of the 2017 NotPetya attack, forensic investigators used network traffic captures to reconstruct exactly how the worm propagated across networks. The PCAP evidence showed the exact moment each machine was compromised, which credentials were used, and how lateral movement occurred. Without the captures — which were running before the attack — the investigation would have been impossible. Continuous network capture is an investment that pays off in forensic capability.

PRACTICE *(2 Hours)*

LAB 7.4.1 — Capture and Analyse Attack Traffic

Objective: Capture a complete attack scenario and analyse the PCAP.

Prerequisites: Kali terminal; Metasploitable2 running; DVWA running

Step 1 — Start Capture Before the Attack

BASH

```
# Start capturing all traffic and save to file sudo tcpdump -i eth0 -w /tmp/week7_capture.pcap & echo "Capture started, PID: $!"
```

Step 2 — Execute an Attack Chain

Perform the following in sequence (the full Week 4 attack chain):

BASH

```
# Step 1: Nmap scan sudo nmap -sS TARGET_IP # Step 2: SQLi against DVWA curl 'http://localhost/dvwa/vulnerabilities/sqli/?id=1+UNION+SELECT+user,password+FROM+users--' # Step 3: SSH brute force (against localhost — lab only) hydra -l root -P /usr/share/wordlists/rockyou.txt -t 4 ssh://localhost
```

Step 3 — Stop Capture

BASH

```
# Stop tcpdump sudo killall tcpdump # Verify capture file ls -lh /tmp/week7_capture.pcap
```

Step 4 — Analyse the PCAP

BASH

```
# Read capture and filter for port scan (SYN packets) sudo tcpdump -r /tmp/week7_capture.pcap 'tcp[tcpflags] & tcp-syn != 0' | head -20 # Find HTTP traffic with SQL keywords sudo tcpdump -r /tmp/week7_capture.pcap -A port 80 | grep -i 'union\|select' # Find SSH connection attempts sudo tcpdump -r /tmp/week7_capture.pcap port 22 | head -30
```

Step 5 — Open in Wireshark

Open Wireshark → File → Open → /tmp/week7_capture.pcap. Apply these display filters one by one and note what you find: tcp.flags.syn==1 && tcp.flags.ack==0, then http.request.uri contains "union", then tcp.port==22.

Step 6 — AI-Assisted Analysis

Export the HTTP stream (Right-click on an HTTP packet → Follow → HTTP Stream → Save). Feed a sanitised excerpt to your AI assistant:

PROMPT

```
I captured this HTTP traffic during a penetration test. Analyse the following request and tell me: what attack type this is, what data the attacker was trying to extract, and what Snort rule would detect this exact pattern. [paste your HTTP stream excerpt here]
```

■ **ANALYSE THIS**

Your PCAP shows the Nmap scan at 08:23:41, the SQLi attack at 08:23:55, and the Hydra SSH brute force starting at 08:24:10. If a SOC analyst was reviewing this capture at 08:24:00 (before the brute force started) — what would they see? What action would they take? Would they be able to prevent the brute force if they acted immediately?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Network Forensics Analysis Report for the attack chain you captured in Lab 7.4.1.

Format this as a professional incident investigation report

Required sections:

- Incident Summary: what happened, when, who was involved
- Evidence: PCAP file details (size, duration, packet count)
- Timeline Reconstruction: using timestamps from PCAP — exact sequence of events
- Attack Phase Analysis: for each phase (scan, SQLi, brute force) — describe what you see in the packets, what the attacker was doing, evidence extracted
- Indicators of Compromise (IoCs): IP addresses, ports, patterns — format as shareable threat intel
- Detection Gaps: at what point could this attack have been stopped? Which defensive control would have prevented each phase?
- Recommendations: 5 specific controls that would have detected or prevented this attack chain

Evidence appendix:

- Include sanitised tcpdump output, Wireshark screenshots (if you have GUI access), Snort/Suricata alert logs

DOCUMENT

Writing Prompt:

Write a Network Capture Retention and Privacy Policy for your organisation.

- Required: What network traffic will be captured, how long it will be retained, who can access captures, how personal data in captures will be handled (employees have privacy rights — their web traffic may be captured), how captures will be secured against unauthorised access
- NDPA alignment: capturing employee internet traffic may constitute processing of personal data — explain how to comply with NDPA 2023 while maintaining security monitoring capability

DAY
5

Review, Assessment & Portfolio

LEARN *(1 Hour) — Week Review*

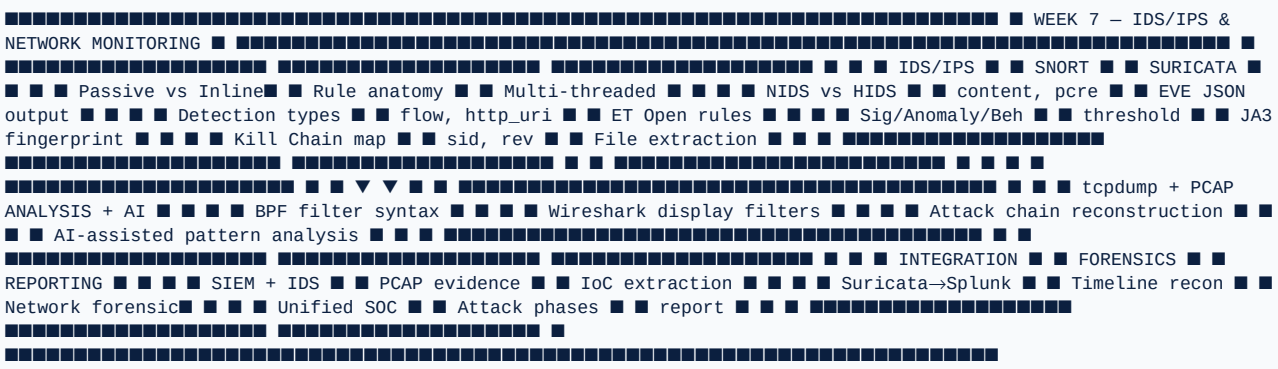
LEARNING OBJECTIVES

1. Demonstrate all Week 7 skills without referencing notes

2. Complete a comprehensive skills assessment

3. Create a professional portfolio entry covering IDS/IPS

Week 7 Mind Map — IDS/IPS and Network Monitoring



PRACTICE *(2 Hours) — Skills Assessment Labs*

LAB 7.5.1 — Snort Rule Writing Challenge

Objective: Write and test Snort rules from memory — no notes, 30-minute limit.

Prerequisites: Snort running; DVWA running; no notes

Write Snort rules for each requirement. All rules must have correct syntax and valid sids.

1. Detect any HTTP GET request containing the string 'passwd' in the URI (directory traversal)
2. Detect more than 15 SYN packets from a single source in 10 seconds (port scan)
3. Detect HTTP traffic where the User-Agent header contains 'sqlmap' (automated SQLi tool)
4. Detect ICMP ping packets larger than 100 bytes (ICMP tunnel/exfiltration)
5. Detect any FTP login attempt (AUTH command) from an external IP

LAB 7.5.2 — PCAP Analysis Under Pressure

Objective: Analyse a pre-captured PCAP and identify the attack — 45 minutes.

Prerequisites: tcpdump and Wireshark available; no notes

Create a test PCAP by running: sudo tcpdump -i eth0 -w test.pcap -c 500. Then run a mix of: normal web browsing (DVWA), Nmap scan, SQLi attempt. Swap your PCAP with a fellow student if possible, or use the one you captured in Day 4.

Answer these questions from the PCAP alone:

1. What is the attacker's IP address?
2. What time did the attack begin?
3. What attack techniques were used? (list at minimum 2)
4. Was any sensitive data transmitted? What was it?
5. Write a tcpdump filter that would have isolated the attack traffic at the time of capture

BUILD *(2 Hours) — Portfolio Entry*

■ MINI PROJECT 3 — Network Security Monitoring Lab

This Mini Project demonstrates your complete network detection capability. It combines Snort, Suricata, tcpdump, and SIEM integration into a unified detection lab.

Deliverables:

- Lab Architecture Diagram: ASCII diagram showing Kali VM, DVWA, Metasploitable, Snort/Suricata sensor, and Splunk — with arrows showing traffic and log flows
- Snort Custom Rules: minimum 8 rules from your Day 2 library, all tested with evidence of alerts firing
- Suricata Deployment: screenshot of Suricata running with ET Open rules; sample EVE JSON alert
- Splunk Integration: SPL queries showing Suricata alerts in your SIEM dashboard
- Attack Chain PCAP Analysis: your Day 4 network forensics report (minimum 5 pages)
- Detection Coverage Matrix: table mapping each rule to MITRE ATT&CK; technique, showing your coverage vs gaps
- Performance Notes: any false positive issues encountered and how you tuned the rules
- BONUS: Write a Snort rule using pcre that detects a Base64-encoded Meterpreter payload pattern in HTTP traffic

Tools: Snort · Suricata · tcpdump · Wireshark · Splunk · Claude AI (documented)

Mini Project 3 — Submission

- Due: End of Week 7 (today)
- Format: PDF report + snort rules file (local.rules) + PCAP file
- Evaluation: Snort rules quality (30%), Suricata deployment (20%), PCAP analysis (25%), Splunk integration (15%), Documentation (10%)

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 7 learning reflection. Length: 300–400 words.

- The most powerful Snort rule you wrote and what specifically it detects
- How does network-based detection (IDS) complement log-based detection (SIEM)? Give a specific scenario where you need both
- What surprised you most about what is visible in network traffic?
- How will the PCAP analysis skills from this week be useful in Week 8 (Digital Forensics)?
- If you were a SOC manager with a £10,000 budget for network detection — Suricata (free) + analyst time, or a commercial IDS like Palo Alto NGFW? Justify your answer

Weekly Checklist (Student-Facing)

- ■ I can explain the difference between IDS and IPS, and NIDS vs HIDS without notes
- ■ I have Snort installed and running in IDS mode on Kali
- ■ I have written at least 8 custom Snort rules and tested them against real attack traffic
- ■ I can explain every component of a Snort rule (action, protocol, IP, port, direction, options)
- ■ I have installed Suricata with the Emerging Threats Open ruleset
- ■ I have forwarded Suricata EVE JSON alerts to Splunk
- ■ I can write tcpdump filters to isolate specific attack traffic
- ■ I have analysed a PCAP and reconstructed an attack chain from packet data
- ■ I have used AI to analyse network traffic and compared its findings to my manual analysis
- ■ My Mini Project 3 is complete and submitted

Skills Verification

Skill	How to Demonstrate It	Pass Standard
IDS vs IPS	Explain difference, give deployment scenario for each	Correct definitions; appropriate scenarios
Snort rule writing	Write a SQLi detection rule from memory	Correct syntax; valid sid; rule fires on attack traffic
Snort deployment	Show Snort running with alerts for Nmap scan	Alert fires; alert message visible in console
Suricata EVE JSON	Show Suricata alert in EVE JSON format	Valid JSON; alert fields correctly populated
tcpdump capture	Capture 100 packets filtering for port 80 only	Correct command; output shows only HTTP packets
PCAP analysis	Identify attack type from provided PCAP in 10 minutes	Correct attack identified; evidence from PCAP cited
SIEM integration	Show Suricata alerts in Splunk dashboard	SPL query returns IDS events; dashboard panel working

Self-Assessment Quiz

Instructions

- No notes — answer from memory.
- Write on separate sheet.
- Key in Instructor Notes.

Conceptual Questions

1. Explain why running Snort in IPS (inline) mode is riskier than IDS mode. Give a specific scenario where a false positive in IPS mode could cause significant business disruption.
2. What is the difference between signature-based and anomaly-based detection? Give one attack that signature-based would catch but anomaly-based would miss, and one that anomaly-based would catch but signature-based would miss.

Rule Analysis

3. Analyse this Snort rule and explain exactly what it detects:

SNORT RULE

```
alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS $HTTP_PORTS ( msg:"Suspicious";
flow:to_server,established; content:"../"; http_uri; nocase; threshold: type both, track by_src,
count 5, seconds 30; sid:1000099; rev:1; )
```

- What attack does this detect?
- What does 'threshold: type both, track by_src, count 5, seconds 30' mean?
- What is a false positive risk for this rule?
- How would you improve it?

4. Write a Snort rule from memory to detect Metasploit Meterpreter traffic — it typically uses HTTP POST requests with a specific user-agent or payload pattern. Explain your design choices.

tcpdump/PCAP Questions

5. Write a tcpdump filter that captures ONLY: outbound traffic from your network (192.168.1.0/24) to external IPs on port 443 (HTTPS), with packet size greater than 1400 bytes. Explain why this specific filter is useful for detecting data exfiltration.
6. You are given a 500MB PCAP from an incident. Describe your analysis workflow — what tools, what filters, what order, what you are looking for. How do you decide what to report as findings?

Integration Questions

7. Describe how you would build a detection pipeline that: captures packets with tcpdump, sends alerts to Snort, forwards Snort alerts to Splunk, and triggers a Splunk alert that pages the SOC analyst. Draw this as a text diagram.

Additional Resources

Snort Resources

- Snort Official Documentation: https://www.snort.org/documents
- Snort Rule Writing Guide: https://www.snort.org/faq/readme-rules
- TryHackMe — Snort Room (Free): https://tryhackme.com/room/snort
- TryHackMe — Snort Challenge: https://tryhackme.com/room/snortchallenges1

Suricata Resources

- Suricata Official Documentation: https://suricata.readthedocs.io
- Emerging Threats Open Ruleset: https://rules.emergingthreats.net
- Suricata Rule Writing Guide: https://suricata.readthedocs.io/en/latest/rules/
- OISF (Open Information Security Foundation): https://oisf.net

Network Analysis

- Wireshark Documentation: https://www.wireshark.org/docs/
- tcpdump Manual: https://www.tcpdump.org/manpages/tcpdump.1.html
- BPF Filter Reference: https://biot.com/capstats/bpf.html
- Malware Traffic Analysis (Free PCAP samples): https://www.malware-traffic-analysis.net

MITRE ATT&CK; for IDS Mapping

- MITRE ATT&CK; Navigator: https://mitre-attack.github.io/attack-navigator/
- Sigma Rules (IDS-to-SIEM translation): https://github.com/SigmaHQ/sigma
- SOC Prime Detection as Code: https://socprime.com

Looking Ahead — Week 8

Preview — What's Coming in Week 8

- Digital Forensics and Incident Response (DFIR) — the final week of Phase 2
- Memory forensics with Volatility — extracting evidence from RAM
- Disk forensics with Autopsy — recovering deleted files and examining artifacts
- Incident Response lifecycle — formal process from detection to closure
- Your Week 7 PCAP captures become forensic evidence in Week 8 investigations

What to Review Before Week 8

- Your Week 7 PCAP captures — you will use these as forensic evidence
- Your Snort alerts — IDS alerts are often the starting point for forensic investigations
- The NIST incident response lifecycle from Week 6 — Week 8 puts it into practice

What to Set Up Before Week 8

- Install Volatility 3: pip3 install volatility3 --break-system-packages
- Install Autopsy: https://www.autopsy.com/download/ (available for Linux)
- Keep your Week 7 PCAP files — they are your forensic evidence for Week 8
- Read NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response

End of Phase 2 — Blue Team & Defence

- Week 8 completes Phase 2. By the end of Week 8 you will have:
- Complete detection stack: SIEM (Splunk) + IDS (Snort/Suricata) + HIDS (Wazuh)
- Web attack detection (Week 5–6) + Network attack detection (Week 7) + Forensics (Week 8)
- Weeks 9–12 cover: Cryptography, Secure Development, Cloud Security, and Capstone